Reviewer Pack — Minimal p-Adic Order of Unitary Groups and DPLL Proof Length…

Entry 0a2cb4010ea4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 08:56:11 UTC

Minimal p-Adic Order of Unitary Groups and DPLL Proof Length Inequality

Entry ID: `0a2cb4010ea4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 08:56:11 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): DPLL Proof Complexity

Statement:

> For every CNF φ with n variables, the minimal order of a unitary group over the p-adic numbers that can generate all possible assignments of φ is bounded by the length of the DPLL proof for φ, such that O(φ) = Θ(|DPLL_proof(φ)|^2).

Rationale (proposer's reasoning):

> p-adic Analysis provides a framework to study algebraic structures in the context of number fields. Unitary groups over p-adic numbers are known to be related to the algebraic structure of solutions to polynomial equations. If this conjecture holds, it would suggest that the complexity of finding satisfying assignments for a CNF can be inferred from the algebraic properties of its solution set, potentially providing new insights into the structure of NP-complete problems.

Taxonomy category: `p-adic_analysis` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `adf5561feefba987`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each CNF with n variables (n ≤ 40), if the Pearson correlation coefficient between the minimal order of a unitary group and DPLL proof length is ≥ 0.8, or if any seed produces a Pearson correlation coefficient < 0.5, the conjecture is supported; otherwise, it is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.6s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_cnf(n):
    cnf = []
    for _ in range(2**n):
        clause = [random.randint(1, n) * (-1 if random.choice([True, False]) else 1) for _ in rang
        cnf.append(clause)
    return cnf

def dpll(cnf, new_literals=[]):
    unit_clause = next((c for c in cnf if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        return dpll([c for c in cnf if -literal not in c], new_literals + [literal])

    if not cnf:
        return True

    literal = next((l for l in range(1, n+1) if all(l not in clause and -l not in clause for claus
    if literal is None:
        return False

    return dpll([c for c in cnf if -literal not in c], new_literals + [literal]) or dpll(cnf, new_

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    cnf = generate_cnf(n)
    length_dpll_proof = dpll(cnf)

    if length_dPLL_proof is None:
        return {
            "metric_name": "O(φ)",
            "metric_value": float('inf'),
```

```
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "DPLL proof length not finite"
        }

    O_phi = Fraction(length_dpll_proof, 2)

    return {
        "metric_name": "O(φ)",
        "metric_value": float(O_phi),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] and r["metric_value"] < 0.5 for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"] and r["m
        print(f"RESULT: FALSIFIED counterexample=\"low correlation\" first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot verify the Pearson correlation coefficient as required by the pre-registered support condition. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15692
2	preregistration	ollama_remote	glm4:latest	0	0	9469
3	novelty	ollama_remote	glm4:latest	0	0	8933
4	novelty	ollama_remote	glm4:latest	0	0	8870
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14766
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10635
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10266
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8895
9	judge	ollama_remote	glm4:latest	0	0	20630

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 108157 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0a2cb4010ea4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a2cb4010ea4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0a2cb4010ea4.tar.gz` (if generated)